写一下CS50学习过程中关于指针和地址部分的笔记和心得，其实就是在下面的注释部分，但是还是整理一下。

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int n = 50;
    char *s = "HI!";
    int *p = &n; // p 是一个 指向int类型变量 的地址 的 整数 8 bytes
    // p 用来存储地址
    printf("%i\n", n);
    printf("%p\n", p); // %p 获取地址 &n
    // p 存储 n的位置 OR p 指向 n的位置
    printf("%i\n", *p);
    // 打印 p所指向的那个东西，De-reference 解引用=
    printf("%p\n", s);
    printf("%s\n", s);
    // %s (String) 不仅仅是打印了个变量值，更是根据你提供的地址找到了TA!
    // 这块一定要理解好，printf不需要找到*s,他只需要找到s，也就是这个字符串的第一位
(s[0])开始的地方，然后遍历到\0就可以了，所以
    // 我们这里没有写 *s，而是直接s。因为*s对应的就是s[0]这么一个单字母了，我们不需要
这个
    printf("%s\n", s + 1);
    // s + 1 就是将起始位置向后移动了一位，于是只会打印出 “I!”
    printf("%c %c\n", s[0], *(s + 0));
    // %c 打印 *s 的字符 就是相当于s[0]，因为s指向的就是这个字符串起始位置，也就是H所
在的位置
    printf("%c %c\n", s[1], *(s + 1));
    printf("%c %c\n", s[2], *(s + 2));
    // 其实类似于 s[i] 的这种 “语法糖” ，在编译时是这样处理的:
    // s[i] --> *(s + i)
    // 不用在乎数据类型的问题，编译器会自动为你换算


    // 所以让我们做点禁忌的事情
    // printf("%c\n", *(s + 20000));
    // Segmentation fault (core dumped)
    // 访问了不该访问的内存

    // strcmp 做了个什么事情呢?
    // 比如 s, t 是两个strings
    // 我们怎么判断他们是否相等?
    // i form 1 to n 判断 *(s + i) ?= *(t + i)
    // strcmp(s, t) 就是帮你干了这么件事情
    // 若是相同的，return 0;
    // 若是字典序 s > t, return 1;
    // 若是字典序 s < t, return -1;

    string m = get_string("m: ");
    string t = get_string("t: ");
```

```
    printf("%p\n", m);
    printf("%p\n", t);
}
```

其实指针部分的主要问题就是C语言对于 * 这个符号的“滥用”。

1.int *p = &n; 此时*的作用是**定义指针**，p 是一个**指向int类型变量的地址**的**整数**，大小为8 bytes。

p 可以理解为存储了**n的位置**，你知道p，也就能找到n，并且n在内存中的位置是确定的，尽管n的数值会改变，但是这个变量所在的位置是在其被定义的那个时候就确定了。

2.printf("%i\n", *p); 此时*的作用就是**De-reference 解引用指针**，寻找到p对应的那个“东西”。

剩下的就是一些细枝末节的东西，例如：

%p就是打印p所指向的**地址** e.g:printf("%p\n", p); 输出的就是0x16b6c710c，也就是n所在的地址。

对于字符串 (char数组) 而言，printf中的%s，你只需要提供你想要打印的起始位置，printf函数会自动替你完成“**往下走，直到\0**”的步骤。

e.g:

```
char *s = "HI!";

printf("%p\n", s); // out:0x16ae1f108
printf("%s\n", s); // out:HI!
printf("%s\n", s + 1); // out:I!
```

我们这里没有写 *s，而是直接写s。因为*s对应的就是s[0]所对应的这么一个单字母(H)了，我们不需要这个，我们需要的是起始位置，也就是起始的地址。

类似于 s[i] 的这种 “语法糖” ，在编译时是这样处理的：**s[i] --> *(s + i)** 不用担心，编译器会为你做好一切，这类语法🍬 还是很好用的。

来点**禁忌**的事情

比如我们写这么一个东西

```
printf("%c\n", *(s + 200000));
```

我们想访问一下 s后200000位内存存着些什么，但是运行的时候就是**Segmentation fault (core dumped)**，你访问了（我访问了/计算机访问了）不该访问的内存。

我们再围绕字符串写一点点东西，比如**strings.h**库中的**strcmp**函数，它的实现原理就是：输入s, t两个strings。然后开始判断 *(s + i) ?= *(t + i)，其中i从0到n遍历。

若是相同的，return 0; 若是字典序 s > t, return 1; 若是字典序 s < t, return -1;

Mostly_Harmless

写于2023.11.16 下午二时